

MS1 – Solutions to Similar Practice Questions

Reference solutions for the newly generated questions. The techniques mirror the supplied patterns: Pandas DataFrame construction/cleaning/grouping, NumPy validation/statistics/categorization/streak scanning, dictionary-based OOP state and exceptions, AVG() OVER(PARTITION BY), and LAG(). The source Pandas pattern requires copies, fixed column order, filtering, grouping, sorting and zero-based indexes. ■filecite■turn0file0■L25-L51■ The source SQL patterns explicitly require window AVG and LAG-based previous-record calculations. ■filecite■turn0file1■L17-L23■ ■filecite■turn0file1■L44-L54■

SOLUTION 1 – Hospital Equipment Service Analyzer

```
import pandas as pd

class HospitalEquipmentAnalyzer:
    def create_services_df(self, service_data: list) -> pd.DataFrame:
        columns = [
            "ServiceID", "EquipmentID", "Department",
            "ServiceDate", "UsageHours", "DowntimeHours",
            "ServiceStatus"
        ]
        return pd.DataFrame(service_data, columns=columns)

    def clean_service_data(self, df: pd.DataFrame) -> pd.DataFrame:
        valid = (
            df.notna().all(axis=1)
            & (df["UsageHours"] > 0)
            & (df["DowntimeHours"] >= 0)
            & df["ServiceStatus"].isin(["Completed", "Scheduled"])
        )

        columns = [
            "ServiceID", "EquipmentID", "Department",
            "ServiceDate", "UsageHours", "DowntimeHours",
            "ServiceStatus"
        ]

        return df.loc[valid, columns].copy().reset_index(drop=True)

    def add_attention_flag(
        self, df: pd.DataFrame, downtime_threshold: float
    ) -> pd.DataFrame:
        result = df.copy()
        result["NeedsAttention"] = (
            result["DowntimeHours"] > downtime_threshold
        ).astype(int)
        return result

    def equipment_performance_summary(
        self, df: pd.DataFrame
    ) -> pd.DataFrame:
        result = (
            df.groupby("EquipmentID", as_index=False)
            .agg(
                ServiceCount=("ServiceID", "count"),
                TotalUsageHours=("UsageHours", "sum"),
                AverageDowntime=("DowntimeHours", "mean")
            )
        )
        result["AverageDowntime"] = result["AverageDowntime"].round(2)
        return result.sort_values("EquipmentID").reset_index(drop=True)

    def low_usage_equipment(
        self, df: pd.DataFrame, usage_threshold: float
    ) -> pd.DataFrame:
        result = (
            df.groupby("EquipmentID", as_index=False)["UsageHours"]
            .sum()
            .rename(columns={"UsageHours": "TotalUsageHours"})
        )
        result = result[result["TotalUsageHours"] < usage_threshold]
        return result.sort_values("EquipmentID").reset_index(drop=True)

    def departmental_service_cost(
        self, df: pd.DataFrame
    ) -> pd.DataFrame:
```

```
completed = df[df["ServiceStatus"] == "Completed"].copy()
completed["ServiceCost"] = completed["DowntimeHours"] * 300.0
result = (
    completed.groupby("Department", as_index=False)["ServiceCost"]
    .sum()
)
return result.sort_values("Department").reset_index(drop=True)
```

SOLUTION 2 – Aquarium Water Quality Analyzer

```
import numpy as np

def create_water_temperature_array(values: list) -> np.ndarray:
    return np.asarray(values, dtype=np.float64).reshape(-1)

def validate_water_temperature_array(arr: np.ndarray) -> bool:
    if arr.size == 0:
        return False
    if not np.issubdtype(arr.dtype, np.number):
        return False
    return bool(np.all((arr >= 12.0) & (arr <= 30.0)))

def compute_water_temperature_stats(arr: np.ndarray) -> tuple:
    return (
        round(float(np.mean(arr)), 2),
        round(float(np.std(arr)), 2),
        round(float(np.max(arr)), 2),
        round(float(np.min(arr)), 2)
    )

def categorize_water_temperatures(arr: np.ndarray) -> np.ndarray:
    return np.select(
        [
            (arr >= 12.0) & (arr <= 18.0),
            (arr > 18.0) & (arr <= 26.0),
            (arr > 26.0) & (arr <= 30.0)
        ],
        ["Cold", "Optimal", "Warm"],
        default="Invalid"
    )

def longest_warm_streak(arr: np.ndarray) -> int:
    current = 0
    longest = 0

    for value in arr:
        if 26.0 < value <= 30.0:
            current += 1
            longest = max(longest, current)
        else:
            current = 0

    return longest
```

SOLUTION 3 – Library Book Loan Registry

```
class LibraryLoanRegistry:
    def __init__(self):
        self.loans = {}

    def register_loan(
        self, loan_id: str, book_title: str, member_name: str
    ) -> dict:
        if loan_id in self.loans:
            raise ValueError("Loan already registered")

        self.loans[loan_id] = {
            "book_title": book_title,
            "member_name": member_name,
            "status": "On Loan"
        }
        return self.loans

    def change_member(
        self, loan_id: str, new_member_name: str
    ) -> dict:
        if loan_id not in self.loans:
            raise KeyError("Loan not found")

        self.loans[loan_id]["member_name"] = new_member_name
        return self.loans

    def get_loan_details(self, loan_id: str) -> dict:
        if loan_id not in self.loans:
            raise KeyError("Loan not found")
        return self.loans[loan_id]

    def loans_by_member(self, member_name: str) -> list:
        return [
            loan_id
            for loan_id, details in self.loans.items()
            if details["member_name"] == member_name
        ]
```

SOLUTION 4 – Average Delivery Time by Route

Filter completed rows before applying the window function, ensuring non-completed deliveries do not affect the average.

```
SELECT
  r.route_name,
  d.driver_name,
  x.delivery_time,
  ROUND(
    AVG(x.delivery_time) OVER (
      PARTITION BY x.route_id
    ),
    2
  ) AS avg_route_time
FROM Deliveries AS x
JOIN Routes AS r
  ON x.route_id = r.route_id
JOIN Drivers AS d
  ON x.driver_id = d.driver_id
WHERE x.status = 'Completed'
ORDER BY
  r.route_name,
  x.delivery_time,
  d.driver_name;
```

Core ideas: JOIN supplies names; WHERE removes non-completed rows; AVG() OVER(PARTITION BY route_id) retains each delivery row while calculating a route-level average; ROUND formats the average; ORDER BY matches the required sequence.

SOLUTION 5 – Days Between Patient Appointments

```
WITH ordered_appointments AS (  
  SELECT  
    p.patient_name,  
    a.appointment_id,  
    a.appointment_date,  
    LAG(a.appointment_date) OVER (  
      PARTITION BY a.patient_id  
      ORDER BY a.appointment_date, a.appointment_id  
    ) AS previous_appointment_date  
  FROM Appointments AS a  
  JOIN Patients AS p  
  ON a.patient_id = p.patient_id  
)  
SELECT  
  patient_name,  
  appointment_id,  
  appointment_date,  
  DATEDIFF(  
    appointment_date,  
    previous_appointment_date  
  ) AS days_since_previous_appointment  
FROM ordered_appointments  
ORDER BY  
  patient_name,  
  appointment_date,  
  appointment_id;
```

Core ideas: PARTITION BY creates a separate sequence per patient; ORDER BY appointment_date, appointment_id makes the sequence deterministic; LAG() obtains the previous date; DATEDIFF() calculates the day gap; the first appointment naturally produces NULL.

SQL dialect note: the solution uses MySQL-style DATEDIFF(). If the assessment environment uses another SQL engine, the date-difference expression may need to be adapted while retaining the same LAG() pattern.